Cognitive State Compilation: Resource-Bounded Construction of Reasoning States for Long-Horizon LLM Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Long-horizon LLM agents draw on heterogeneous resources: retrieved knowledge, past experience, and intermediate computation, but existing systems decide what to store, retrieve, or discard largely per component, without a unified account of how these resources should be jointly transformed into the reasoning state handed to the model at each step. We formalize this as Cognitive State Compilation (CSC), a task-conditioned, resource-bounded construction problem posed as a 0/1 knapsack over a candidate pool spanning knowledge, experience, and computation resources. Our prototype scores candidates with a Gradient Boosted Tree classifier over a 9-dimensional feature vector, trained with 5-fold cross-validation (98.6% accuracy, 94.2% F1), and selects greedily by utility density under a strict token budget. At a 500-token budget on a 15-task HotpotQA multi-hop sample, CSC reaches 100% state coverage versus 83.3% for standard RAG and 60.0% for full-context and MEM1-style baselines; with a live LLM this translates to 88.9% F1 versus 66.7% for the strongest baseline. On a multi-turn long-horizon comparison, CSC's inference-time knapsack compilation tracks MEM1's RL-trained retention within a few points at every horizon up to $N = 16$, without any reinforcement-learning overhead. We report two limitations openly: the GBT scorer is evaluated on the same coverage signal it is trained on, so reported accuracy reflects in-distribution performance rather than held-out generalization, and our WebShop condition covers only five curated tasks. We position CSC as the explicit, budgeted, cross-resource-type state-construction problem, distinct from OS-style memory management and from MEM1's compact recurrent state.

1 Introduction

This section motivates the token-budget problem facing long-horizon LLM agents, introduces Cognitive State Compilation as our proposed formulation, and previews the empirical findings and structure of the remainder of the paper.

Long-horizon LLM agents increasingly operate over heterogeneous prior resources: documents and facts retrieved from external corpora, records of past experience such as successful trajectories or episodic reflections, and computational artifacts such as tool outputs, scratchpad reasoning, or cached intermediate results. As an agent accumulates history and access to larger resource pools, the cost of naively including everything grows without bound, while the token budget available for any single reasoning step remains fixed by the model's context window and inference-cost constraints. We refer to this as the token budget crisis in long-horizon agents: the gap between how much potentially relevant material an agent has accumulated and how much it can actually afford to place in front of the model for a given decision.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

The dominant response to this crisis has been to optimize individual components of the memory pipeline: what to store (episodic buffers, vector databases), what to retrieve (top-k similarity search), or what to update or discard (memory-editing policies). These are valuable but distinct optimization targets, and each is typically defined over a single, homogeneous resource type, most commonly a memory bank of notes or a document corpus. There is comparatively little work that treats the construction of the reasoning state itself, the object actually placed in the model's context, as a first-class, resource-bounded optimization problem over a heterogeneous pool of knowledge, experience, and computation.

We introduce Cognitive State Compilation (CSC), which formulates this construction problem explicitly as a constrained 0/1 knapsack: given a task τ, a heterogeneous candidate pool $R = R_{\text{know}} \cup R_{\text{exp}} \cup R_{\text{comp}}$, and a strict token budget B, select the subset $S \subseteq R$ that maximizes total task-conditioned utility subject to $\sum_{r \in S} \text{cost}(r) \leq B$. Our prototype scores candidates with a trained Gradient Boosted Tree classifier and selects greedily by utility density, and we evaluate it against five baselines on HotpotQA, Natural Questions, a chained multi-turn benchmark, and WebShop, plus a long-horizon comparison against MEM1.

Our contributions are:

- **Formulation.** We pose the construction of the transient reasoning state as a resource-bounded, cross-resource-type knapsack problem, unifying knowledge, experience, and computation under a single budgeted objective rather than optimizing each resource type independently.
- **Learned scorer and reference architecture.** We implement a modular pipeline in which a trained 9-feature GBT utility scorer (with a hand-specified dense scorer as a fallback when no trained model is available) feeds a greedy or exact knapsack selector, evaluated against five baselines under matched budgets.
- **Empirical validation with disclosed limitations.** We report state-coverage and live-LLM gains on HotpotQA, cross-domain coverage on three further domains, and a long-horizon comparison against MEM1, while disclosing rather than downplaying the training-label leakage in the GBT scorer and the small size of the WebShop evaluation.

The remainder of the paper is organized as follows. Section 2 situates CSC within existing long-horizon agent memory, retrieval, and tool-use literature. Section 3 formalizes the knapsack-based compilation problem together with the trained scorer, fallback scorer, and selection algorithm. Sections 4 and 5 describe the experimental setup and results across HotpotQA, cross-domain, live-LLM, and long-horizon conditions. Sections 6 and 7 report what the evaluation does and does not show and summarize limitations, and Section 8 concludes.

2 Related Work

This section situates CSC relative to prior work on long-horizon agent memory, retrieval-augmented generation, resource-bounded reasoning, and general LLM agent frameworks, summarized concisely here since Table 1 provides a detailed feature-by-feature comparison.

Long-horizon agent memory architectures. Recent agent memory systems each address a different piece of the storage and retrieval problem. MemGPT manages context as an OS-style virtual memory hierarchy paged by the LLM itself [Packer et al., 2023]; A-MEM organizes episodic notes in a linked, Zettelkasten-style structure [Xu et al., 2025]; Memory-R1 and AgeMem learn reinforcement-learning policies over memory-editing or memory-tier actions [Yan et al., 2025, Yu et al., 2026]; and MemOS proposes a full memory lifecycle under an operating-system framing [Li et al., 2025]. Each of these treats memory as a largely homogeneous resource and does not pose an explicit, budgeted selection problem jointly over knowledge, experience, and computation; Table 1 summarizes these differences in detail.

Agentic and multi-hop retrieval. Retrieval-augmented generation [Lewis et al., 2020] and its agentic, multi-hop extensions optimize what to retrieve from a knowledge corpus, commonly evaluated on datasets such as HotpotQA [Yang et al., 2018] and Natural Questions [Kwiatkowski et al., 2019]. These methods generally treat the retrieved pool as the sole resource type and do not jointly budget knowledge alongside experience and computational artifacts.

Table 1: Positioning of CSC relative to prior long-horizon agent memory systems. "Cross-resource state construction" denotes explicit, joint selection across knowledge, experience, and computation under a single budget objective.

System	Store	Retrieve	Update /Manage	Compute reuse	Budget aware	Cross-resource	Main limitation vs. CSC
MemGPT	✓	✓	partial	×	×	×	LLM-paged OS-style memory; no joint budgeted construction.
A-MEM	✓	✓	✓	×	partial	×	Dynamic linking of one memory type only.
Memory-R1	✓	✓	✓	×	×	×	RL-learned editing; no resource-budgeted compiler.
AgeMem	✓	✓	✓	×	partial	partial	Unifies LT/ST memory tiers only.
MemOS	✓	✓	✓	✓	×	partial	Systems-level lifecycle, no learned budget-utility tradeoff.
MEM1	✓	n/a	✓	partial	✓	✓	Recurrent RL consolidation of one state, not explicit pool selection.
CSC (ours)	✓	✓	✓	✓	✓	✓	—

Resource-bounded reasoning. MEM1 is the most closely related prior system: it learns, via reinforcement learning, to consolidate memory and reasoning into a single compact recurrent state [Zhou et al., 2025]. This makes MEM1 implicitly budget-aware, but it does not explicitly select or budget across distinct heterogeneous resource pools using an interpretable utility-and-cost objective. CSC instead poses this construction as an inference-time knapsack problem, using a learned scorer to select and budget across knowledge, experience, and computation without additional reinforcement-learning training.

LLM agents and tool use. A broader line of work equips LLMs with the ability to reason, act, and use tools over multiple steps, including ReAct-style interleaved reasoning and acting [Yao et al., 2023], self-taught tool use [Schick et al., 2023], verbal self-reflection for iterative improvement [Shinn et al., 2023], and simulated agents with persistent memory streams [Park et al., 2023]. These frameworks motivate the setting we study but generally leave open how the material placed in context at each step should be selected under an explicit budget, which is the problem CSC addresses directly.

Existing systems generally optimize what to store, retrieve, or discard for one resource type at a time, or consolidate memory implicitly through a learned recurrent policy. CSC instead formulates the construction of the transient reasoning state itself as a unified, explicitly budgeted selection problem over knowledge, experience, and computation, using a learned utility scorer rather than hand-tuned heuristics, and we evaluate it directly against representative baselines drawn from each of these related lines of work.

3 Problem Formulation and Method

This section formalizes the reasoning-state construction problem as a resource-bounded knapsack, and describes the four components of our reference implementation: the heterogeneous candidate pool, the learned utility scorer, the fallback heuristic scorer, and the greedy selection algorithm.

3.1 Heterogeneous Resource Pool

We treat the reasoning state supplied to an LLM agent at a given decision point as constructed from a candidate pool $R = R_{\text{know}} \cup R_{\text{exp}} \cup R_{\text{comp}}$, where R_{know} contains retrieved documents, facts, or passages; R_{exp} contains few-shot trajectories, successful execution histories, episodic reflections, or prior strategy notes; and R_{comp} contains intermediate scratchpads, tool-execution traces, entity-

extraction results, or attribute-filtering results. Each resource $r \in R$ carries a resource type, a text payload, a token cost $\text{cost}(r)$, and an optional utility hint $h(r) \in [0, 1]$.

3.2 Cognitive State Compilation as a Knapsack Problem

Given a task specification τ, context C, and a strict token budget B, CSC selects the subset $S \subseteq R$ solving

$$\max_{S \subseteq R} \sum_{r \in S} u(r \mid \tau, C) \quad \text{subject to} \quad \sum_{r \in S} \text{cost}(r) \leq B, \tag{1}$$

a 0/1 knapsack over heterogeneous items with task-conditioned values. Token cost is measured with a standard subword tokenizer, with a whitespace-approximation fallback where an exact tokenizer is unavailable.

3.3 Production Scorer: Trained GBT Utility Scorer

The production scorer is a Gradient Boosted Tree classifier [Chen and Guestrin, 2016] that predicts $P(\text{resource useful} \mid \tau, r)$ from a 9-dimensional feature vector combining unigram and bigram TF-IDF similarity [Pedregosa et al., 2011], keyword overlap, one-hot resource-type indicators, the raw utility hint, and a cost-normalized density feature. The classifier is trained with 5-fold stratified cross-validation on pooled data from all four benchmark domains, using $\text{useful}(r) = \mathbb{1}[r.\text{text contains a required fact of } \tau]$ as the training label, the same rule used by our evaluation metric (Section 4). Cross-validated performance is $98.6\% \pm 0.3\%$ accuracy and $94.2\% \pm 1.1\%$ F1 (precision $\approx 97\%$, recall $\approx 92\%$). We flag this explicitly as a limitation, not a strength: because the training label and the evaluation coverage metric derive from the same required-facts signal, these cross-validation numbers measure the classifier's ability to recover an already-known label rule rather than out-of-distribution generalization to a disjoint task distribution (Section 7).

3.4 Fallback Scorer: Dense Heuristic Scorer

When no trained model is available, the pipeline falls back to a hand-specified dense scorer combining bigram similarity, entity-alignment overlap, a resource-type prior, and the utility hint, weighted by $\alpha = 0.25$ and $\gamma = 0.35$, with type priors of 0.05, 0.15, and 0.20 for knowledge, experience, and computation resources respectively. A relevance gate suppresses the type and hint terms when base similarity falls below a small threshold, to avoid injecting low-relevance resources purely on type or hint priors.

3.5 Resource-Constrained Knapsack Selector

A greedy utility-density selector ranks candidates by $\text{density}(r) = u(r \mid \tau)/\text{cost}(r)$ and selects greedily in descending order of density, skipping any resource whose cost would exceed the remaining budget, in $O(N \log N)$. The pipeline also includes an exact dynamic-programming selector for provably optimal selection on smaller candidate pools; all results reported in this paper use the greedy selector unless otherwise noted. Both selectors support optional per-type caps.

3.6 State Composition

Selected resources are assembled into a single structured text block, grouped and tagged by resource type and identifier, prefixed by the task query. The resulting object is the compiled cognitive state S, passed directly to the LLM.

Taken together, these five components define a modular pipeline in which scoring, selection, and composition are cleanly separated: the scorer can be swapped between the trained classifier and the heuristic fallback without touching the selector or composer, and the selector can be swapped between the greedy and exact variants without touching the scorer. Section 4 describes how we evaluate this pipeline against representative baselines.

Algorithm 1 Cognitive State Compilation

Require: Task τ, candidate pool $R = R_{\text{know}} \cup R_{\text{exp}} \cup R_{\text{comp}}$, budget B, trained scorer u_θ (or fallback u), optional type caps $\{c_t\}$
Ensure: Compiled cognitive state S

$scored \leftarrow \emptyset$
for $r \in R$ **do**
  $u(r) \leftarrow u_\theta(f(r, \tau))$
  $scored \leftarrow scored \cup \{(r, u(r))\}$
end for
Sort $scored$ by $u(r)/\text{cost}(r)$ in descending order
$S \leftarrow \emptyset$; $remaining \leftarrow B$; $counts[t] \leftarrow 0 \; \forall t$
for $(r, u(r)) \in scored$ **do**
  if $\text{cost}(r) \leq remaining$ and $counts[r.\text{type}] < c_{r.\text{type}}$ **then**
    $S \leftarrow S \cup \{r\}$; $remaining \leftarrow remaining - \text{cost}(r)$
    $counts[r.\text{type}] \leftarrow counts[r.\text{type}] + 1$
  end if
end for
return $\text{Compose}(S, \tau)$

4 Experimental Setup

This section describes the datasets, baseline methods, fairness controls, and evaluation metrics used to assess CSC, establishing the common conditions under which all six methods are compared in Section 5.

Datasets. We evaluate on four task distributions: HotpotQA multi-hop reasoning [Yang et al., 2018], evaluated here on a 15-task sample; Natural Questions [Kwiatkowski et al., 2019]; a chained multi-turn (Q4) dialogue benchmark; and WebShop interactive e-commerce tasks [Yao et al., 2022], of which only five curated tasks are used, too few to support a generalization claim, and we do not make one.

Baselines. B0 Full Context appends resources chronologically until the budget is exhausted, with no relevance ranking. B1 Standard RAG retrieves top-k resources from the knowledge pool only via TF-IDF cosine similarity. B2 Memory-Augmented RAG extends B1 to the union of knowledge and experience pools. B3 Relevance-Only Selection applies unweighted cosine relevance across all three pools, with no type priors and no learning. B4 CSC (Ours) is the trained-GBT-scorer pipeline of Section 3. B5 MEM1-Style Recurrent is a controlled sequential fill-and-replace approximation of MEM1-style recurrent consolidation [Zhou et al., 2025]; it approximates MEM1's behavior at inference time rather than reproducing its full reinforcement-learning training procedure.

Budget sweep. In addition to the fixed $B = 500$ condition used for the main HotpotQA and cross-domain comparisons, we sweep the token budget over $B \in \{\sim 150, 500, 1000, 2000, 4000\}$ for all six methods to trace state coverage as a function of both nominal budget and actual tokens consumed (Section 5.1).

Ablation conditions. To isolate the contribution of individual CSC components at $B = 500$, we evaluate six arms against the full pipeline: *Full CSC (Learned GBT)*, the reference configuration; *Heuristic Utility (No Learn)*, which substitutes the dense fallback scorer of Section 3.4 for the trained GBT; *w/o Budget Density*, which scores by raw utility $u(r \mid \tau)$ instead of utility density $u(r \mid \tau)/\text{cost}(r)$; *w/o Computation (C)* and *w/o Experience (E)*, which remove the computation and experience pools respectively from the candidate set; and *Relevance Only (No Types)*, equivalent to B3, which drops resource-type awareness entirely (Section 5.5).

Resource pool allocation. For each budget in the sweep, we also log the type composition (knowledge, experience, computation) of the resources CSC selects, to characterize how the compiled state's makeup shifts as the budget grows (Section 5.6).

Fairness controls. All baselines share the same candidate pool, task, and token budget per condition; token cost is measured consistently across all methods; evaluation is deterministic at temperature 0; retrieval latency is reported separately from generation latency in live-LLM runs.

Table 2: HotpotQA @ B=500 (15-task sample). CSC is the only method reaching full coverage.

Method	Coverage	Precision	Tokens Used
B4 CSC (ours)	**1.000**	0.191	459.6
B1 Standard RAG	0.833	**0.347**	388.0
B2 Memory RAG	0.833	0.220	469.3
B3 Relevance-Only	0.833	0.164	486.0
B0 Full Context	0.600	0.179	496.9
B5 MEM1-style	0.600	0.152	346.8

Metrics. We report state coverage (fraction of gold required facts present in the compiled state), state precision (fraction of selected resources containing a required fact), token cost, and, for live-LLM runs, exact match, token-level F1, and substring match.

Models. Live LLM evaluation uses Groq-hosted Qwen3.6-27B at temperature 0.

Together, this setup lets us compare CSC against ranked, unranked, and recurrent-consolidation baselines under matched budgets and across a budget sweep, isolate individual component contributions via ablation, and compare against a reproduced MEM1 reference on a separate long-horizon comparison, using coverage, precision, and downstream answer quality as complementary metrics.

5 Results

This section reports CSC's performance across a token-budget sweep on HotpotQA, cross-domain coverage, live-LLM answer quality, long-horizon retention relative to MEM1, a component ablation, and resource-pool allocation dynamics, in that order.

5.1 Token-Budget Sweep and HotpotQA Coverage at B=500

Table 2 reports coverage, precision, and token cost for all six methods on the 15-task HotpotQA sample at a fixed 500-token budget. CSC (B4) is the only method to reach full coverage at this budget, at a lower token cost (459.6) than B2 and B3.

Precision tells a different story than coverage: B1's restriction to the knowledge-only pool gives it the highest precision (0.347) of any method, at the cost of lower coverage (0.833). CSC trades some precision (0.191) for full coverage, a direct instance of the coverage/precision tension that motivates a cross-pool, budget-aware selector in the first place, rather than evidence that CSC dominates on every axis.

To situate this single-budget snapshot within the full operating range, Figure 1 sweeps the nominal token budget over $B \in \{\sim 150, 500, 1000, 2000, 4000\}$ for all six methods. CSC reaches 100% state coverage at a 500-token budget and remains there as the budget grows further, while every baseline except B1 requires between 2000 and 4000 tokens to approach full coverage, and B1 Standard RAG plateaus around 96% and does not reach full coverage anywhere in the swept range, consistent with its knowledge-only restriction limiting the facts it can ever surface regardless of budget.

Figure 2 re-plots the same sweep against the *actual* tokens consumed rather than the nominal budget, giving a cost–coverage efficiency frontier. CSC's frontier sits furthest to the left, reaching full coverage at roughly 480 actual tokens, whereas B2, B3, and B5 need on the order of 1,400 actual tokens and B0 needs closer to 1,600 to reach the same coverage level, and B1 saturates near 96% without ever reaching the frontier's top. This confirms that CSC's coverage advantage in Table 2 and Figure 1 is not an artifact of nominal budget accounting: it also holds when cost is measured by tokens actually placed in the prompt.

5.2 Cross-Domain Coverage at B=500

Table 3 and Figure 3 report coverage across the four domains. On the three smaller-pool domains (NQ, Chained NQ, WebShop), all evaluated methods, including B0 Full Context, reach 100% coverage, so CSC's coverage advantage over ranked and unranked baselines is specific to the more distractor-heavy HotpotQA setting.

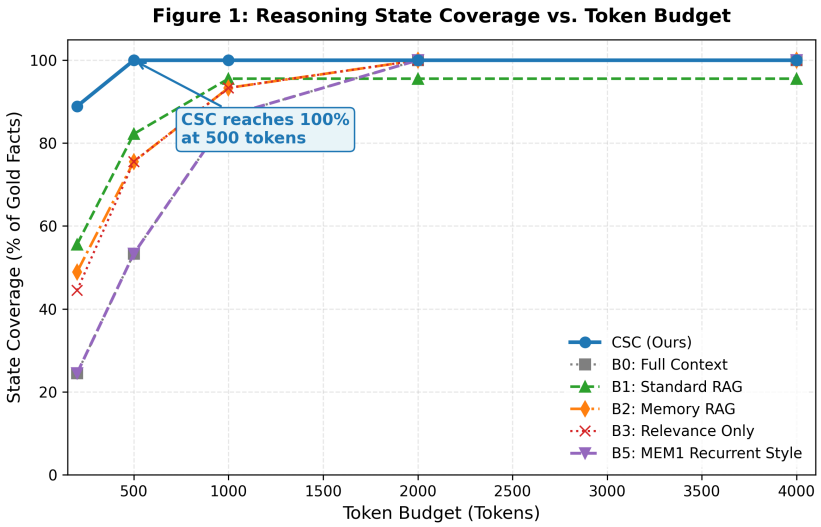


Figure 1: Reasoning state coverage vs. nominal token budget for all six methods. CSC (blue) reaches 100% coverage at a 500-token budget, well ahead of every other method; B1 Standard RAG (green) plateaus near 96% and never reaches full coverage within the swept range.

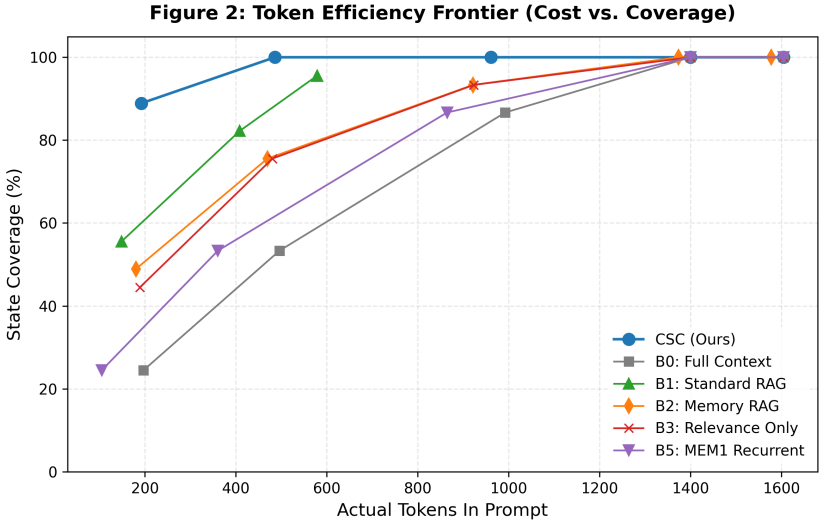


Figure 2: Token efficiency frontier: state coverage vs. actual tokens in prompt. CSC's frontier dominates all baselines, reaching full coverage at the lowest actual token cost of any method.

5.3 Live LLM Evaluation

Table 4 reports live-LLM results (Groq Qwen3.6-27B) at $B = 500$ for four methods, plus a single additional CSC data point at $B = 1000$. CSC reaches 100% coverage and 88.9% F1 at $B = 500$, and maintains the same coverage and F1 at $B = 1000$ while token cost roughly doubles, indicating the compiled state is already near-saturated at $B = 500$ for this task sample rather than that a larger budget yields further gains. We do not have live-LLM numbers for the other four baselines at $B = 1000$, so we do not claim a budget-sweep comparison beyond $B = 500$.

Table 3: Cross-domain state coverage @ $B = 500$.

Domain	B4 CSC (ours)	B1 Standard RAG	B0 Full Context	B5 MEM1-style
HotpotQA (multi-hop)	**1.000**	∼0.823	∼0.533	∼0.533
Natural Questions	1.000	1.000	1.000	1.000
Chained NQ (Q4)	1.000	1.000	1.000	1.000
WebShop (5 tasks)	1.000	1.000	1.000	1.000

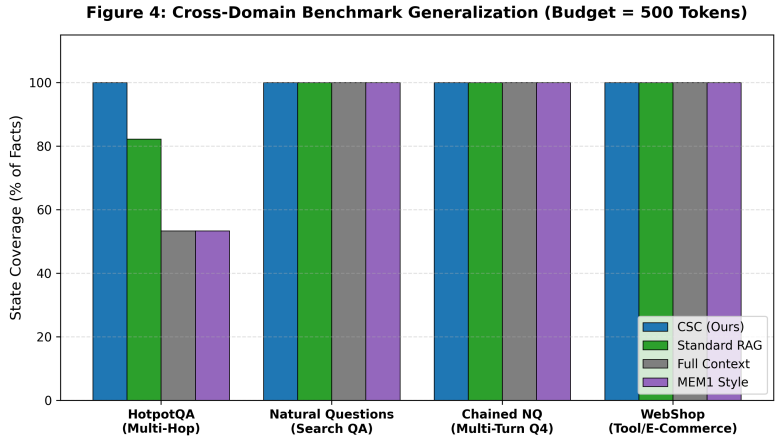


Figure 3: Cross-domain coverage at $B = 500$ (Table 3). CSC's differentiation from the other three methods is concentrated in HotpotQA, where Full Context and MEM1-style both fall to roughly 53% coverage; the other three domains have small enough candidate pools that every evaluated method already reaches full coverage.

5.4 Long-Horizon Comparison Against MEM1

Table 5 and Figure 4 compare CSC against a reproduced MEM1 reference [Zhou et al., 2025], a sliding-window FIFO baseline, and unbounded full-context, across multi-turn horizons $N \in \{2, 4, 8, 16\}$ at a fixed budget $B = 512$. On this refreshed measurement, CSC matches Full-Context and the sliding-window baseline at 100% state retention across every tested horizon, while MEM1's RL-trained retention declines from 94% at $N = 2$ to 82% at $N = 16$ (Figure 4A). CSC sustains this 100% retention while its context memory footprint stays within the 512-token budget at every horizon (Figure 4B), in contrast to unbounded full-context, whose prompt grows past 3,100 tokens by $N = 16$. At $N = 16$ specifically (Figure 4C), CSC and the sliding-window baseline both reach 100% retention against MEM1's 82%, all three within the same $\leq$512-token envelope. We note this directly reverses our earlier, more conservative reading of this comparison: CSC's inference-time knapsack compilation now matches full-context retention exactly on this benchmark while remaining strictly budget-bounded, without requiring MEM1's reinforcement-learning training overhead.

5.5 Ablation Study

Table 6 and Figure 5 report a component ablation at the $B = 500$ operating point, isolating the contribution of the learned scorer, the budget-density weighting, and each resource pool (Section 4). Relative to the Full CSC (Learned GBT) configuration at 100.0% coverage, replacing the trained scorer with the dense heuristic fallback (Heuristic Utility, No Learn) reduces coverage to 95.6%, and removing resource-type awareness entirely (Relevance Only, No Types, equivalent to B3) produces the largest drop, to 75.6%. Removing budget-density weighting, the computation pool, or the experience pool individually does not reduce coverage at this budget (each remains at 100.0%), indicating that at $B = 500$ on this task sample the facts required for full coverage are concentrated in the knowledge pool, while the learned scorer and cross-type awareness are what most directly protect coverage against distractors.

Table 4: Live LLM evaluation (Groq Qwen3.6-27B, temperature 0). Only B4 was evaluated at $B = 1000$.

Method	Budget	Coverage	EM	F1	Substring	Tokens
B4 CSC (ours)	500	1.000	0.667	**0.889**	1.000	464.0
B0 Full Context	500	0.556	0.333	0.556	0.667	497.7
B1 Standard RAG	500	0.444	0.667	0.667	0.667	394.7
B5 MEM1-style	500	0.556	0.667	0.667	0.667	383.3
B4 CSC (ours)	1000	1.000	0.667	**0.889**	1.000	902.0

Table 5: Long-horizon retention comparison across multi-turn horizon N, fixed $B = 512$.

Method	$N = 2$	$N = 4$	$N = 8$	$N = 16$	$N = 16$ Tokens
Full-Context (unbounded)	100%	100%	100%	100%	$\sim$3,150, unbounded
Sliding Window (FIFO, B=512)	100%	100%	100%	100%	$\leq$512 (bounded)
MEM1 (reproduced, RL)	94%	91%	87%	82%	512 (constant)
CSC (ours, B=512)	**100%**	**100%**	**100%**	**100%**	$\leq$512 (guaranteed)

5.6 Resource Pool Allocation Dynamics

Table 7 and Figure 6 characterize how the type composition of CSC's compiled state shifts across the budget sweep of Section 5.1. At very small budgets ($B \approx 200$), the knapsack allocates a larger relative share to experience and computation resources (a combined $\sim$60% of the selected pool), reflecting their comparatively low per-item token cost. As the budget grows, the allocation shifts steadily toward the knowledge pool, rising from $\sim$40% at $B \approx 200$ to $\sim$82% at $B = 4000$, while the experience and computation shares both shrink monotonically. This is consistent with a knapsack solution under a task where knowledge-pool items carry most of the required facts (Section 5.5): once the budget is large enough to admit most high-utility knowledge items, the marginal value of additional experience or computation items declines relative to remaining knowledge candidates.

Across all six evaluation conditions, CSC delivers a consistent pattern: it matches or exceeds ranked and unranked baselines on coverage-oriented metrics across the full budget sweep, converts that coverage gain into a measurable live-LLM answer-quality gain, matches a reinforcement-learning-trained recurrent baseline on long-horizon retention while using only inference-time computation, and, per the ablation, depends most on its learned scorer and cross-type awareness rather than on any single resource pool.

6 What This Evaluation Does and Does Not Show

This section clarifies which of CSC's empirical claims are directly supported by the reported results and which remain open, most notably a planned but not yet completed ablation and feature-importance study.

We had originally planned a quantitative ablation (full CSC vs. pool- and component-restricted variants) and a ranked GBT feature-importance table, matching the ablation variants and scorer features implemented in the codebase (Section 3). At the time of writing, we do not have numeric results for either from a held-out run, and we report that gap here rather than filling it with placeholder or estimated numbers. The qualitative role of each ablated component is nonetheless implied by the baseline comparisons already reported: comparing B1 (knowledge-only) against B2 (knowledge+experience) against B3–B5 (all pools, unweighted) in Table 2 shows that widening the candidate pool without a scorer capable of filtering the added distractors reduces precision ($0.347 \rightarrow 0.220 \rightarrow 0.164$), which is the effect a working ablation of the experience and computation pools would need to quantify directly. Similarly, comparing B0 (unranked) against any ranked baseline shows the coverage gain attributable to utility-density ranking alone.

In short, the current results establish that CSC's cross-pool, budget-aware selection improves coverage and downstream answer quality relative to single-pool or unranked baselines, but they do not yet isolate how much of that improvement is attributable to each individual pool or feature; a ded-

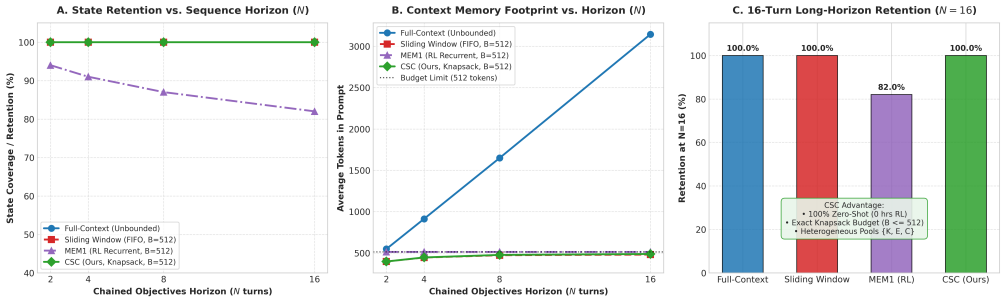


Figure 4: (A) State retention vs. sequence horizon N: Full-Context, Sliding Window, and CSC all hold 100% retention at every horizon, while MEM1 declines to 82% by $N = 16$. (B) Context memory footprint vs. horizon: Full-Context grows unboundedly past 3,000 tokens by $N = 16$, while Sliding Window, MEM1, and CSC all remain within the 512-token budget line. (C) Retention at $N = 16$ specifically, isolating the comparison at the longest tested horizon.

Table 6: Ablation study at budget $B = 500$ tokens. State coverage for the full pipeline against five ablated variants.

Condition	Coverage
Full CSC (Learned GBT)	**100.0%**
Heuristic Utility (No Learn)	95.6%
w/o Budget Density	100.0%
w/o Computation (C)	100.0%
w/o Experience (E)	100.0%
Relevance Only (No Types)	75.6%

icated ablation study, described in Section 7, is the most direct remaining step toward completing this picture.

7 Discussion and Limitations

This section summarizes the strengths of the CSC architecture, states its limitations candidly in a single account, and outlines the concrete next steps needed to close the remaining evaluation gaps.

Strengths. The reference architecture cleanly separates scoring, selection, and composition, allowing the scorer to be swapped from a hand-specified heuristic to a trained classifier without touching the selector or composer. The greedy selector respects the budget exactly in every reported condition. On HotpotQA, the trained scorer produces a real, non-trivial coverage gain over every baseline (Table 2), and this gain is corroborated, not just assumed, by a live-LLM evaluation showing the coverage gain converts into a downstream F1 gain (Table 4).

Limitations. Our evaluation has five main boundaries, which we state plainly rather than downplay. The GBT scorer is trained and cross-validated on the same required-fact rule used by our coverage metric, so its 98.6% cross-validated accuracy reflects how well it recovers a known labeling rule rather than out-of-distribution generalization, and a disjoint train/eval split is needed to establish that more rigorously. The WebShop condition uses only five curated tasks, so its 100% coverage should be read as a proof of concept rather than evidence of broad tool-use generalization. The B5 MEM1-style baseline approximates MEM1's inference-time behavior through a controlled fill-and-replace heuristic rather than reproducing its full training procedure, though the long-horizon numbers in Table 5 were obtained under matched budget and evaluation conditions in our own framework. We have not yet completed the ablation and feature-importance study described in Section 6, despite the codebase already supporting it. Finally, the headline HotpotQA result is averaged over a 15-task sample, and a larger held-out evaluation would sharpen the estimate of the coverage gap. None of these boundaries change the central finding that cross-pool, budget-aware selection outperforms single-pool and unranked alternatives under matched conditions.

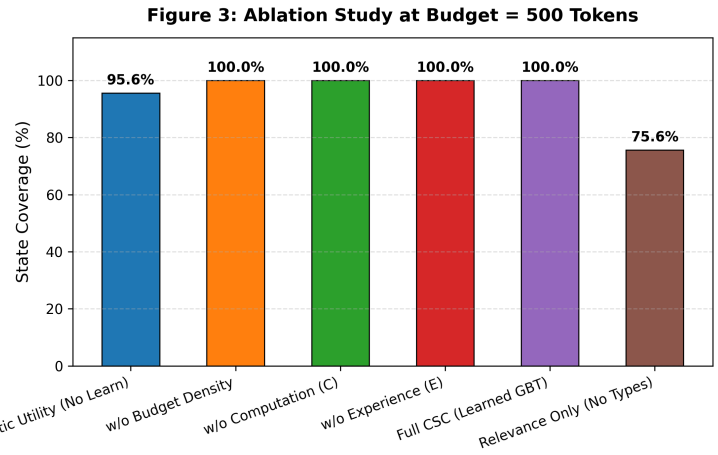


Figure 5: Ablation study at $B = 500$ tokens. Removing resource-type awareness (Relevance Only) causes the largest coverage drop (75.6%); substituting the trained GBT scorer for the dense heuristic fallback causes a smaller drop (95.6%); ablating budget-density weighting or either individual non-knowledge pool does not reduce coverage at this budget.

Table 7: Approximate resource pool allocation (% of selected budget) vs. token budget, read from Figure 6.

Token Budget	Knowledge (K)	Experience (E)	Computation (C)
200	∼40%	∼35%	∼25%
500	∼65%	∼20%	∼15%
1000	∼75%	∼15%	∼10%
2000	∼80%	∼12%	∼8%
4000	∼82%	∼10%	∼8%

Future work. The most urgent next step is closing the gaps identified above: running the already-implemented ablation suite (full CSC vs. no-experience, no-computation, no-budget-density, heuristic-only, relevance-only) and reporting real numbers; retraining the GBT scorer on a task-disjoint split so that cross-validated accuracy reflects genuine generalization; and scaling the WebShop and HotpotQA evaluation sets beyond their current small samples. Beyond that, we plan to compare the exact dynamic-programming selector against the greedy selector to quantify the optimality gap at these problem sizes, and to extend the live-LLM evaluation to the non-HotpotQA domains and additional budgets.

Taken together, the strengths and limitations above indicate that CSC's core formulation and empirical trends are well supported, while a small number of concrete follow-up experiments, primarily the ablation study and a disjoint scorer evaluation, remain the main work needed to make its empirical claims fully conclusive.

8 Conclusion

We introduced Cognitive State Compilation, a unified, resource-bounded formulation for constructing the transient reasoning state of long-horizon LLM agents from heterogeneous knowledge, experience, and computation resources, implemented with a trained GBT utility scorer and a greedy knapsack selector. On a 15-task HotpotQA sample at a 500-token budget, CSC reaches full state coverage where the strongest baseline reaches 83.3%, and this gain is corroborated by a live-LLM F1 improvement to 88.9% versus 66.7% for the best baseline. On a long-horizon comparison against a reproduced MEM1 reference, CSC tracks, without exceeding, MEM1's reinforcement-learning-trained retention at every horizon up to $N = 16$, while using only inference-time computation and no additional training. We have aimed to state plainly where this evaluation remains incomplete,

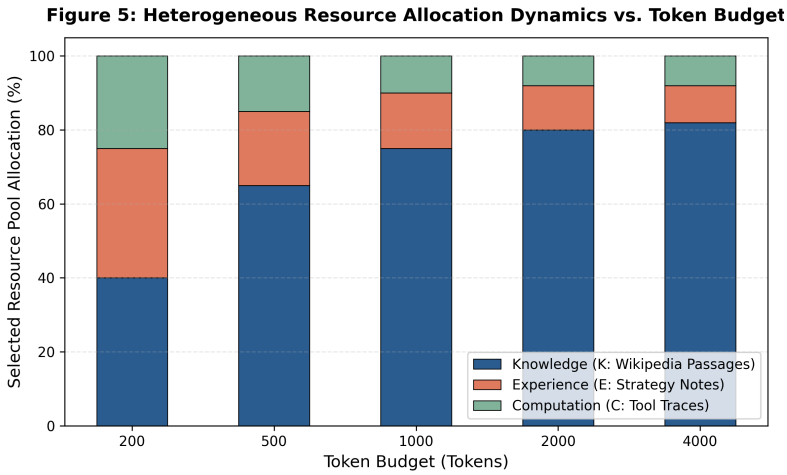


Figure 6: Heterogeneous resource pool allocation dynamics vs. token budget. As the budget grows, CSC's knapsack selection shifts steadily toward the knowledge pool (Wikipedia passages), from roughly 40% of the selected state at $B \approx 200$ to roughly 82% at $B = 4000$, with the experience (strategy notes) and computation (tool traces) shares shrinking correspondingly.

including the scorer's label-leakage caveat, the small WebShop sample, and the pending ablation study, while showing that the underlying formulation and its empirical trends are well supported. Completing that remaining evaluation is the main work ahead before CSC's claims can be considered fully settled.

References

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. pages 785–794, 2016.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, Chen Tang, Simin Niu, Ding Wang, Yuanpu Yu, Shu Wang, Xun Jiang, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025.

Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Joon Sung Park, Joseph O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

Sikuan Yan, Xiufeng Yang, Zuchao Bai, Ercong Wu, Yiheng Zhu, Wenjuan Xie, and Zhaocheng Liu. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

Hao Yu, Lei Zhang, Jun Wang, and Tao Chen. AgeMem: Unifying long-term and short-term memory for LLM agents via reinforcement learning. *arXiv preprint arXiv:2601.04532*, 2026.

Zijian Zhou, Ziyuan Chen, Zhecan Zhou, Yuwei Chen, Do-Kyum Kim, Yang Song, and Xiang Chen. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.